

Traitement des Données Multimodales

PRÉTRAITEMENT DES DONNÉES

2 juillet 2026

Prof. Abdellah Adib

`abdellah.adib@fstm.ac.ma`





Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Positionnement du chapitre
Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes
Pratiques

Contact Information

1

Introduction

Positionnement du chapitre
Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes Pratiques

Contact Information

- ◆ La structuration du prétraitement repose sur une logique modale en trois temps :

- ◆ **Nettoyer** les données brutes (artefacts, bruit, valeurs manquantes),
- ◆ **Transformer** pour normaliser la représentation,
- ◆ **Encoder** pour rendre les données exploitables par un modèle.

- ◆ **Des modalités hétérogènes, des techniques spécialisées**

Texte, image et audio nécessitent chacun un pipeline de prétraitement adapté : tokenisation et lemmatisation pour le texte, filtrage et égalisation pour l'image, framing et fenêtrage pour l'audio.

- ◆ **Un objectif commun**

Tous les pipelines convergent vers un même but : produire des **représentations numériques normalisées** exploitables par les algorithmes d'apprentissage, qu'il s'agisse de vecteurs de tokens, de tenseurs image ou de matrices de frames audio.

Vue d'ensemble du Prétraitement Multimodal

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Positionnement du chapitre 3
Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes
Pratiques

Contact Information

♦ Trois modalités, trois pipelines complémentaires

Chaque pipeline suit une progression logique depuis la donnée brute jusqu'au descripteur prêt pour l'apprentissage.

Modalité	Étapes clés du pipeline	Bibliothèques Python
Texte	Nettoyage → Tokenisation → Stop-words → Normalisation → Encodage	NLTK, re, sklearn
Image	Resize → Niv. gris → Bruit → CLAHE → Contours	OpenCV, Pillow, NumPy
Audio	Rééch. → Pré-emphase → Framing → Windowing → VAD	Pydub, Librosa, SciPy



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Positionnement du chapitre
Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes
Pratiques

Contact Information

4

Introduction

Positionnement du chapitre

Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes Pratiques

Contact Information

45

Pourquoi Prétraiter les Données? (1/2)

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Positionnement du chapitre
Logique de prétraitement

Classes d'Outils
Synthèse et Bonnes
Pratiques
Contact Information

5

- ◆ La donnée brute est rarement exploitable directement :
 - ✧ Un fichier texte brut contient de la ponctuation, des majuscules et des mots vides.
 - ✧ Une image peut être sous-exposée ou bruitée.
 - ✧ Un signal audio capte des parasites et varie en fréquence d'échantillonnage.
- ◆ Aucun algorithme ne peut tirer parti de ces données sans **transformation préalable**.

45

Pourquoi Prétraiter les Données? (2/2)

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Positionnement du chapitre
Logique de prétraitement

Classes d'Outils

Synthèse et Bonnes
Pratiques

Contact Information

6

- ◆ Chaque modalité a ses propres artefacts :
 - ✧ Texte : encodages mixtes, contractions, valeurs manquantes.
 - ✧ Image : bruit gaussien, faible contraste, dimensions hétérogènes.
 - ✧ Audio : fond sonore, fréquence d'échantillonnage variable, silences non labellisés.
D'où la nécessité d'un **pipeline dédié par modalité**.
- ◆ L'impact sur la performance est décisif :
 - ✧ Un prétraitement rigoureux améliore directement la précision des modèles ML,
 - ✧ Réduit les temps d'entraînement et garantit la **reproductibilité** des résultats.
- ◆ C'est la première étape de tout pipeline industriel de données.

45

Arbre de Décision — Choisir son Pipeline

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Positionnement du chapitre
Logique de prétraitement

Classes d'Outils

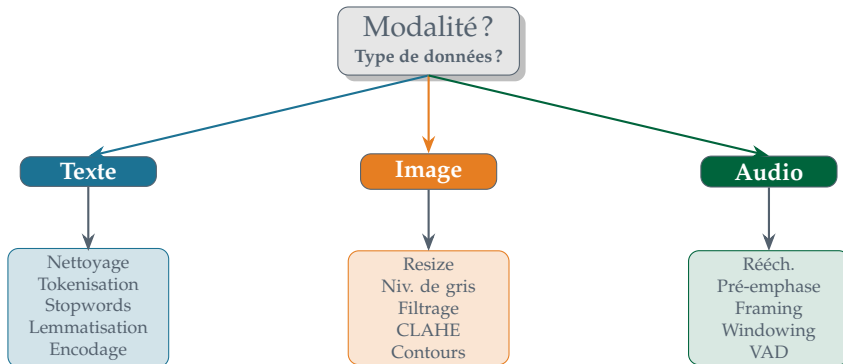
Synthèse et Bonnes
Pratiques

Contact Information

7

◆ Principe de sélection

Le choix du pipeline de prétraitement se base sur la **nature** de la modalité et sur l'objectif applicatif.



45



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données

Textuelles

Traitement des Données

Visuelles

Traitement des Données

Audio

Synthèse et Bonnes

Pratiques

Contact Information

8

Introduction

Classes d'Outils

Traitement des Données Textuelles

Traitement des Données Visuelles

Traitement des Données Audio

Synthèse et Bonnes Pratiques

Contact Information

45



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

9

Introduction

Classes d'Outils

Traitement des Données Textuelles

Traitement des Données Visuelles

Traitement des Données Audio

Synthèse et Bonnes Pratiques

Contact Information

45



[Texte] Classe 1

★ Prétraitement des Données ★ Textuelles

NLTK · re · sklearn

Principe Fondamental

Traitement Automatique du Langage (TAL)

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

11

◆ Définition

Le **Traitement Automatique du Langage** (TAL) regroupe l'ensemble des techniques permettant à une machine d'analyser, comprendre et transformer du texte brut en représentations numériques exploitables.

◆ Utilités

Dans les projets multimodaux, il s'applique aux sous-titres, transcriptions, annotations et commentaires associés aux données image et audio.

◆ Pipeline TAL

- ✧ **Nettoyage** (casse, ponctuation, chiffres)
- ✧ **Tokenisation** (mots, phrases)
- ✧ **Stopwords** (filtrage)
- ✧ **Normalisation** (stemming / lemmatisation)
- ✧ **Encodage** (TF-IDF, embeddings)
- ✧ **EDA** (nuages de mots, fréquences).

45

Fonctionnalités clés du TA

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

12

Étape	Description	Exemple
Nettoyage	Suppression casse, ponctuation, caractères spéciaux	<code>re.sub(), .lower()</code>
Tokenisation	Découpage en mots, sous-mots ou phrases	<code>word_tokenize()</code>
Filtrage	Retrait des stop words non informatifs	<code>stopwords.words('french')</code>
Normalisation	Stemming / lemmatisation — forme canonique	<code>SnowballStemmer</code>
Encodage	Représentation numérique pour modèles ML	<code>TfidfVectorizer()</code>

45

Bibliothèques Python pour le TAL (1/2)

Niveaux et règle de choix

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils
Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

13

♦ Un écosystème structuré en trois niveaux complémentaires

Du texte brut au flux de tokens normalisés, ces bibliothèques assurent le nettoyage, le découpage et la réduction morphologique avant toute représentation numérique.

[Niveau 1] Prétraitement linguistique

- ✧ **NLTK** : tokenisation, stopwords (25+ langues), stemming, lemmatisation, POS tagging
- ✧ **re** : nettoyage fin — ponctuation, chiffres, balises HTML
- ✧ **string** : manipulation bas-niveau — `str.lower()`, `str.strip()`

45

◆ Niveau 2 — Vectorisation classique

Elles convertissent les tokens en vecteurs numériques prêts pour les algorithmes ML.

[Niveau 2] Vectorisation classique

- ✧ **scikit-learn** : `TfidfVectorizer`, `CountVectorizer`, `OneHotEncoder`
- ✧ **Gensim** : `Word2Vec`, `FastText`, `GloVe` — vecteurs sémantiques

◆ Niveau 3 — Embeddings contextuels

Ces modèles pré-entraînés capturent le sens d'un mot selon son contexte dans la phrase.

[Niveau 3] Embeddings contextuels

- ✧ **Transformers** (HuggingFace) : `BERT`, `RoBERTa`, `CamemBERT` (français)
- ✧ Vecteur contextuel : un même mot diffère selon la phrase environnante
- ✧ Accès via `AutoModel.from_pretrained('camembert-base')`

◆ Installation & importation

```

pip install nltk scikit-learn gensim
# Ressources NLTK (à exécuter une seule fois)

import nltk, re
nltk.download('punkt')           # tokenisation
nltk.download('stopwords')       # listes de mots vides
nltk.download('wordnet')         # lemmatisation WordNet
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import
    TfidfVectorizer
    
```


Pourquoi combiner NLTK et scikit-learn ?

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

16

♦ Des rôles distincts et complémentaires

Leurs rôles sont complémentaires : NLTK traite la langue, scikit-learn produit les vecteurs ML.

♦ NLTK — Prétraitement linguistique

NLTK transforme le texte brut en tokens propres et normalisés via la tokenisation, le filtrage des stopwords, la racinisation et la lemmatisation.

♦ scikit-learn — Vectorisation et pipeline ML

scikit-learn convertit les tokens en vecteurs numériques (TfidfVectorizer, CountVectorizer, OneHotEncoder) intégrables directement dans tout pipeline ML.

♦ En résumé

NLTK nettoie et normalise → scikit-learn vectorise et modélise. Ensemble, ils constituent un pipeline TAL **industriellement éprouvé** sans aucune dépendance lourde.

45

Nettoyage et Tokenisation

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

17

♦ Ces étapes constituent la première phase obligatoire avant toute opération linguistique.

Étape	Technique et fonction Python
Suppression caractères spéciaux	<code>re.sub(r'[^a-zA-Z0-9\s]', '', texte)</code> Élimine ponctuation, symboles et balises HTML
Mise en minuscules	<code>texte.lower()</code> — uniformise la casse Évite les doublons ("Le" vs "le")
Gestion des nombres	Remplacer : <code>re.sub(r'\d+', 'NUM', texte)</code> ou supprimer selon le contexte applicatif
Tokenisation en mots	<code>word_tokenize(texte, language='french')</code> Gère contractions et mots composés
Tokenisation en phrases	<code>sent_tokenize(texte, language='french')</code> Utile pour l'analyse de sentiment par phrase
Valeurs manquantes	<code>df['texte'].dropna()</code> ou <code>.fillna("")</code> Doublons : <code>df.drop_duplicates(subset='texte')</code>

45

Stopwords, Normalisation et Encodage

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

18

◆ Ces étapes produisent les représentations numériques exploitables par les modèles ML.

Fonctionnalité	Technique et fonction Python
Filtrage stopwords	<code>stopwords.words('french')</code> — 157 mots Extensible : <code>sw.update(['multimodal', 'donnee'])</code>
Racinement (Stemming)	<code>SnowballStemmer("french").stem(token)</code> <code>PorterStemmer()</code> pour l'anglais
Lemmatisation	<code>WordNetLemmatizer().lemmatize(token, pos='v')</code> Plus précise que le stemming; respecte le sens
Encodage TF-IDF	<code>TfidfVectorizer(max_features=5000)</code> Pondère selon la fréquence et la rareté du terme
Embeddings (Word2Vec, BERT)	<code>gensim.models.Word2Vec(sentences, vector_size=100)</code> BERT : <code>transformers.AutoModel.from_pretrained()</code>
EDA — Fréquences & nuages	<code>FreqDist(tokens).most_common(20)</code> <code>WordCloud(width=800).generate(texte)</code>

45

Sommaire Prétraitement Textuel

01 Principe Fondamental
Pipeline TAL, bibliothèques

02 Installation
NLTK, sklearn, gensim

03 Nettoyage des Données
Casse, ponctuation, regex

04 Tokenisation
Mots, phrases, contractions

05 Stopwords & Normalisation
Filtrage, stemming, lemmatisation

06 Valeurs Manquantes
dropna, fillna, déduplication

07 Encodage & Embeddings
TF-IDF, Word2Vec, BERT, GloVe

08 EDA Texte
Fréquences, nuages de mots, N-grammes

[rapport_traitement_multimodal.pdf/Chapitre 1](#)



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données
Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

20

Introduction

Classes d'Outils

Traitement des Données Textuelles

Traitement des Données Visuelles

Traitement des Données Audio

Synthèse et Bonnes Pratiques

Contact Information

45



★ [Image] Classe 2

Prétraitement des Données Visuelles

OpenCV · Pillow · NumPy

♦ Utilités

Le pré-traitement visuel prépare les images brutes pour les modèles d'apprentissage automatique : photographies, scans médicaux, images satellitaires et frames vidéo doivent être standardisés en termes de dimensions, de plage de valeurs et de contraste.

♦ Bibliothèques principales

OpenCV est le standard industriel pour la vision par ordinateur (plus de 2500 algorithmes optimisés). **Pillow** (PIL Fork) complète OpenCV pour la gestion des formats et les opérations haut niveau. **NumPy** assure les opérations matricielles sur les tenseurs image. **scikit-image** apporte des algorithmes spécialisés supplémentaires.

♦ Pipeline Vision

Acquisition (imread) → **Redimensionnement** (resize) → **Conversion** (gris / HSV) → **Normalisation** (/255) → **Filtrage bruit** (Gaussien) → **Amélioration contraste** (CLAHE) → **Détection contours** (Canny) → **Visualisation** (imshow).

◆ Installation & importation

```
pip install opencv-python pillow scikit-image
pip install opencv-contrib-python # SIFT, SURF,
etc.
import cv2
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
# Verification version
print(cv2.__version__)
```

◆ Pourquoi OpenCV + Pillow?

- ❖ OpenCV assure les traitements bas-niveau (filtrage, morphologie, détection) via des fonctions C++ optimisées, directement compatibles NumPy.
- ❖ Pillow apporte le support de formats supplémentaires (TIFF multi-pages, ICO, WebP) et des manipulations haut niveau.
- ❖ Leur combinaison couvre **l'ensemble des besoins image** du module.

♦ Standardisation géométrique et radiométrique

Ces étapes garantissent l'homogénéité des entrées pour tout modèle CNN.

Opération	Technique et fonction Python
Redimensionnement	<code>cv2.resize(img, (224,224), interpolation=cv2.INTER_CUBIC)</code> Standardise les dimensions pour les modèles CNN (224×224, 299×299)
Transformations géométriques	<code>cv2.warpAffine(img, M, (w,h))</code> : rotation, translation <code>cv2.flip(img, 1)</code> : symétrie ; data augmentation
Conversion niveaux de gris	<code>cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)</code> $Y = 0.114B + 0.587G + 0.299R$ (formule ITU)
Conversion LAB	<code>cv2.cvtColor(img, cv2.COLOR_BGR2HSV)</code> Segmentation par couleur (HSV), distance perceptive (HSV/LAB)
Normalisation [0,1]	<code>img_norm = img.astype(np.float32) / 255.0</code> Uniformise la magnitude — indispensable avant tout modèle ML

Filtrage, Amélioration de Contraste et Détection de Contours

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données

Textuelles

Traitement des Données
Visuelles

Traitement des Données
Audio

Synthèse et Bonnes
Pratiques

Contact Information

25

♦ Débruitage, rehaussement et extraction de caractéristiques

Ces techniques améliorent la qualité intrinsèque de l'image et extraient les structures visuelles porteuses d'information.

Opération	Technique et fonction Python
Réduction de bruit Gaussien	<code>cv2.GaussianBlur(img, (5,5), sigmaX=0)</code> Lisse et supprime le bruit haute-fréquence avant traitement
Filtre médian (bruit S&P)	<code>cv2.medianBlur(img, 5)</code> Efficace contre le bruit sel-et-poivre ; préserve les bords
Égalisation d'histogramme (CLAHE)	<code>clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))</code> Améliore localement le contraste ; évite la sur-saturation globale
Détection de bords (Canny)	<code>cv2.Canny(gray, threshold1=100, threshold2=200)</code> Extrait les contours significatifs ; indispensable pour l'OCR
Visualisation	<code>plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))</code> Toujours convertir BGR→RGB avant <code>plt.imshow()</code>

45

Sommaire Prétraitement Visuel

01 Principe Fondamental
Pipeline vision, bibliothèques

02 Installation OpenCV
opencv-python, Pillow, scikit-image

03 Redimensionnement
Resize, warp, flip, interpolation

04 Conversion & Normalisation
Gris, HSV, LAB, /255

05 Réduction de Bruit
Gaussien, médian, bilatéral

06 Amélioration du Contraste
Égalisation, CLAHE, histogramme

07 Détection de Contours
Canny, Sobel, Laplacian

08 Visualisation
imshow, BGR→RGB, Matplotlib

[rapport_traitement_multimodal.pdf/Chapitre 2](#)



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données

Textuelles

Traitement des Données

Visuelles

Traitement des Données

Audio

Synthèse et Bonnes
Pratiques

Contact Information

27

45

Introduction

Classes d'Outils

Traitement des Données Textuelles

Traitement des Données Visuelles

Traitement des Données Audio

Synthèse et Bonnes Pratiques

Contact Information

★ [Audio] Classe 3

Prétraitement des Données Audio

Pydub · Librosa · SciPy

♦ Utilités

Le prétraitement audio transforme des signaux bruités et hétérogènes en représentations normalisées exploitables : enregistrements vocaux, signaux physiologiques, bandes-son de vidéos et podcasts nécessitent chacun une chaîne de traitement adaptée avant toute reconnaissance ou analyse.

♦ Bibliothèques principales

Pydub gère les opérations haut niveau (chargement multi-format, découpage, normalisation de volume). **Librosa** est la référence pour l'analyse spectrale (MFCC, spectrogramme, ZCR, RMS). **SciPy** (`scipy.signal`) assure le filtrage numérique (Butterworth, pré-emphase). **PyAudio** permet la capture temps réel.

♦ Pipeline Audio

Chargement (`from_file`) → **Rééchantillonnage** (16 kHz) → **Pré-emphase** (filtre passe-haut) → **Framing** (25 ms) → **Windowing** (Hamming) → **VAD** (détection activité vocale) → **Normalisation** (z-score / amplitude).

◆ Installation & importation

```
pip install pydub librosa scipy pyaudio
# FFmpeg requis pour MP3, M4A, OGG, FLAC
from pydub import AudioSegment
from pydub.silence import split_on_silence
import librosa
import numpy as np
from scipy import signal
from sklearn.preprocessing import StandardScaler
```

◆ Complémentarité Pydub / Librosa

- ✧ **Pydub** excelle dans les opérations haut-niveau (formats, découpage, volume, export).
- ✧ **Librosa** est spécialisé dans l'analyse spectrale (MFCC, spectrogramme mel, ZCR, RMS, onset detection). **SciPy** prend en charge les filtres numériques précis (Butterworth IIR/FIR).
- ✧ Cette **complémentarité** couvre l'intégralité du pipeline audio ML.

Rééchantillonnage, Pré-emphase et Framing

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données

Textuelles

Traitement des Données

Visuelles

Traitement des Données

Audio

Synthèse et Bonnes

Pratiques

Contact Information

31

♦ Standardisation temporelle et conditionnement du signal

Ces étapes préparent le signal brut pour les descripteurs spectraux.

Étape	Technique et fonction Python
Rééchantillonnage	<code>audio.set_frame_rate(16000).set_channels(1)</code> Théorème Nyquist : $f_e \geq 2f_{max}$ — 16 kHz pour ASR
Pré-emphase	<code>scipy.signal.lfilter([1, -0.97], 1, samples)</code> Amplifie les hautes fréquences ; améliore le rapport S/B
Framing (segmentation)	Taille = 25 ms = 400 éch. à 16 kHz ; pas = 10 ms <code>frames = [s[i:i+400] for i in range(0, N-400, 160)]</code>
Windowing (fenêtrage)	<code>np.hamming(400)</code> — fenêtre de Hamming standard ASR Réduit les discontinuités bord de frame avant FFT
Rééchantillonnage Librosa	<code>librosa.resample(y, orig_sr=44100, target_sr=16000)</code> Plus précis que Pydub sur de grands rapports de fréquence

45

Windowing, VAD et Normalisation

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Traitement des Données

Textuelles

Traitement des Données

Visuelles

Traitement des Données

Audio

Synthèse et Bonnes

Pratiques

Contact Information

32

♦ Filtrage, détection d'activité et normalisation des features

Ces étapes assurent la qualité et la généralisabilité des descripteurs extraits.

Étape	Technique et fonction Python
Fonctions de fenêtrage	<code>np.hamming(L)</code> (ASR), <code>np.hanning(L)</code> (musique) <code>np.blackman(L)</code> (suppression lobes latéraux FFT)
Crossing Rate Crossing Rate	<code>librosa.feature.zero_crossing_rate(y)</code> Distingue parole voisée / non-voisée / silence
VAD — Énergie RMS	<code>librosa.feature.rms(y=y)</code> Détection segments de silence (<code>split_on_silence</code> Pydub)
Réduction de bruit	<code>scipy.signal.butter(4, fc/(fs/2), 'low') + filtfilt</code> MMSE estimator (<code>logmmse</code>) pour débruitage spectral
Normalisation (z-score)	<code>StandardScaler().fit_transform(features)</code> Réduit la variabilité inter-locuteurs ; généralise mieux

45

Sommaire Prétraitement Audio

- | | |
|--|---|
| 01 Principe Fondamental
Pipeline audio, théorème Nyquist | 05 Framing
Segmentation 25 ms, pas 10 ms |
| 02 Installation
Pydub, Librosa, SciPy, PyAudio | 06 Windowing
Hamming, Hanning, Blackman |
| 03 Rééchantillonnage
16 kHz mono, Librosa resample | 07 Détection d'Activité Vocale
ZCR, RMS, auto-corrélation |
| 04 Pré-emphase & Filtrage
lfilter, Butterworth, passe-haut | 08 Normalisation
z-score, MMSE, amplitude |

[rapport_traitement_multimodal.pdf/Chapitre 3](#)



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Pipeline Multimodal
Complet

Travaux Dirigés et Travaux
Pratiques

Contact Information

34

Introduction

Classes d'Outils

Synthèse et Bonnes Pratiques

Pipeline Multimodal Complet

Travaux Dirigés et Travaux Pratiques

Contact Information

45



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Pipeline Multimodal
Complet

Travaux Dirigés et Travaux
Pratiques

Contact Information

35

Introduction

Classes d'Outils

Synthèse et Bonnes Pratiques

Pipeline Multimodal Complet

Travaux Dirigés et Travaux Pratiques

Contact Information

45



[Synthèse] Bilan

★ Pipeline Multimodal Complet

Texte · Image · Audio → Modèle ML

Pipeline de Prétraitement Multimodal

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Pipeline Multimodal
Complet

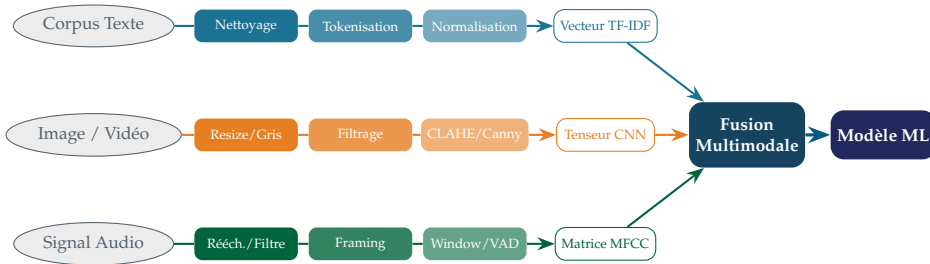
Travaux Dirigés et Travaux
Pratiques

Contact Information

37

♦ Trois chaînes parallèles convergeant vers la fusion

Chaque modalité est prétraitée indépendamment; les vecteurs résultants sont fusionnés pour alimenter un modèle ML unique.



45

Tableau Comparatif des Techniques de Prétraitement

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Pipeline Multimodal
Complet

Travaux Dirigés et Travaux
Pratiques

Contact Information

38

♦ Synthèse pédagogique — 3 modalités

Vue unifiée des techniques, outils et métriques de qualité par modalité.

Modal.	Technique	Objectif	Outil Python	Sortie
Texte	Nettoyage	Supprimer artefacts (casse, ponct.)	<code>re, str.lower()</code>	str nettoyée
	Tokenisation	Découper en unités lexicales	<code>nltk.tokenize</code>	list[str]
	Normalisation	Stemming / lemmatisation	<code>SnowballStemmer</code>	tokens normalisés
	Encodage	Représentation numérique	<code>TfidfVectorizer</code>	matrice sparse
Image	Resize + Gris	Standardiser dimensions et canal	<code>cv2.resize/cvtColor</code>	ndarray (H,W)
	Normalisation	Plage [0, 1] uniforme	<code>img / 255.0</code>	float32
	CLAHE	Amélioration contraste local	<code>cv2.createCLAHE</code>	ndarray amélioré
	Contours	Extraction structures visuelles	<code>cv2.Canny</code>	ndarray binaire
Audio	Rééchantillonnage	Standardiser f_e à 16 kHz	<code>pydub/librosa</code>	AudioSegment
	Pré-emphase	Accentuer hautes fréquences	<code>scipy.signal.lfilter</code>	array numpy
	Framing + Window	Stationnarité locale	<code>numpy</code>	matrice (N, L)
	Normalisation z	Réduire variabilité locuteur	<code>StandardScaler</code>	features normalisées

45

◆ Texte : rigueur linguistique

- ✧ Toujours appeler `nltk.download()` avant usage
- ✧ Préciser `language='french'` explicitement
- ✧ Vérifier l'encodage UTF-8 avant traitement
- ✧ Préférer la lemmatisation au stemming pour le français
- ✧ Inspecter les valeurs manquantes et doublons avant vectorisation

◆ Image : précision et format

- ✧ Convertir BGR→RGB avant tout affichage Matplotlib
- ✧ Normaliser `/255.0` avant tout modèle CNN
- ✧ Préférer CLAHE à `equalizeHist` (local vs global)
- ✧ Utiliser PNG sans perte pour les images de référence

Bonnes Pratiques Générales (2/2)

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction
Classes d'Outils
Synthèse et Bonnes
Pratiques
Pipeline Multimodal
Complet
Travaux Dirigés et Travaux
Pratiques
Contact Information

40

♦ Audio : standardisation du signal

- ✧ Toujours convertir en **mono 16 kHz** avant ASR
- ✧ Appliquer la pré-emphase en premier sur le signal brut
- ✧ Toujours fenêtrer (Hamming) avant FFT ou MFCC
- ✧ Sauvegarder les paramètres (taille frame, f_c) en JSON

♦ Reproductibilité pipeline

- ✧ Fixer `np.random.seed(42)` dès l'import
- ✧ Versionner `requirements.txt` du projet
- ✧ Tester chaque étape du pipeline isolément
- ✧ Relancer Kernel + Run All avant toute remise Jupyter

45



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Pipeline Multimodal
Complet

Travaux Dirigés et Travaux
Pratiques

Contact Information

41

Introduction

Classes d'Outils

Synthèse et Bonnes Pratiques

Pipeline Multimodal Complet

Travaux Dirigés et Travaux Pratiques

Contact Information

45

♦ Trois exercices progressifs (durée : 1h30)

[Texte] Exercice 1

Pipeline TAL complet

- ✦ Charger `article.txt` UTF-8
- ✦ Nettoyage regex + minuscule
- ✦ Tokeniser (NLTK french)
- ✦ Supprimer stopwords fr
- ✦ Lemmatiser (SnowballStemmer)
- ✦ Top-10 fréquences + nuage

Livrable : dict + image
PNG

[Image] Exercice 2

Prétraitement image ML

- ✦ Charger `portrait.jpg`
- ✦ Resize 224×224
- ✦ Convertir en niveaux de gris
- ✦ Appliquer CLAHE (clip=2.0)
- ✦ Détecter contours (Canny)
- ✦ Afficher avant/après

Livrable :
`clahe_canny.png`

[Audio] Exercice 3

Prétraitement audio ASR

- ✦ Charger `discours.wav`
- ✦ Rééchantillonner 16 kHz mono
- ✦ Pré-emphase ($\alpha = 0.97$)
- ✦ Framing 25 ms / pas 10 ms
- ✦ Windowing (Hamming)
- ✦ Calculer ZCR et RMS

Livrable : tableau NumPy
($N, 400$)

- **Données** : `article.txt` (≥ 500 mots), `portrait.jpg`, `discours.wav` (≥ 30 s) fournis par l'enseignant.
- **Env.** : Jupyter + Python 3.9+.

♦ Énoncé : Prétraitement d'un corpus audiovisuel sous-titré

À partir d'un extrait vidéo `conf.mp4` et de son fichier de sous-titres `sous_titres.txt`, construire un pipeline de prétraitement multimodal complet couvrant les trois modalités.

[Étapes] Pipeline à réaliser

- ❖ **Étape 1 (Texte/NLTK)** : nettoyage, tokenisation, stopwords fr, lemmatisation, top-10 mots
- ❖ **Étape 2 (Image/OpenCV)** : extraire 1 frame/resize 224×224, CLAHE, détection Canny
- ❖ **Étape 3 (Audio/Pydub+SciPy)** : mono 16 kHz, pré-emphase, framing Hamming, ZCR/RMS
- ❖ **Étape 4 (Matplotlib)** : figure 3 axes — top-10 mots, frame CLAHE, waveform filtrée

[Critères] Réussite

- ❖ Notebook `.ipynb` propre, commenté
- ❖ Dict. fréquences ≥ 10 mots
- ❖ Dossier `frames/` ≥ 5 images CLAHE
- ❖ Tableau NumPy de forme $(N, 400)$
- ❖ Figure `rapport_mm.png` (3 axes)
- ❖ Aucune cellule en erreur

Données : `conf.mp4` (2–5 min.),



Sommaire

Traitement des
Données
Multimodales

Prof. Abdellah Adib

Introduction

Classes d'Outils

Synthèse et Bonnes
Pratiques

Contact Information

44

Introduction

Classes d'Outils

Synthèse et Bonnes Pratiques

Contact Information

45

Vos retours m'intéressent (commentaires, suggestions, signalements de bogues).
Vous pouvez me joindre aux coordonnées ci-dessous.

Prof. Abdellah Adib

`abdellah.adib@fstm.ac.ma`

Département Informatique
Faculté des Sciences & Techniques Mohammed VI
Université Hassan II de Casablanca